S10 UI Toolkit (optional)

CRE132 worksheet

What you'll make

S03's HUD again (health bar top left, Restart button in the middle, score top right), looking the same but built with **UI Toolkit** instead of a Canvas. UI Toolkit works like a web page: a **UXML** file for the layout, a **USS** file for the looks, and C# for the logic.

This sample is optional. The rest of the course uses the Canvas UI from S03, but it's worth seeing the other system Unity offers. Unity's own editor is built with UI Toolkit.

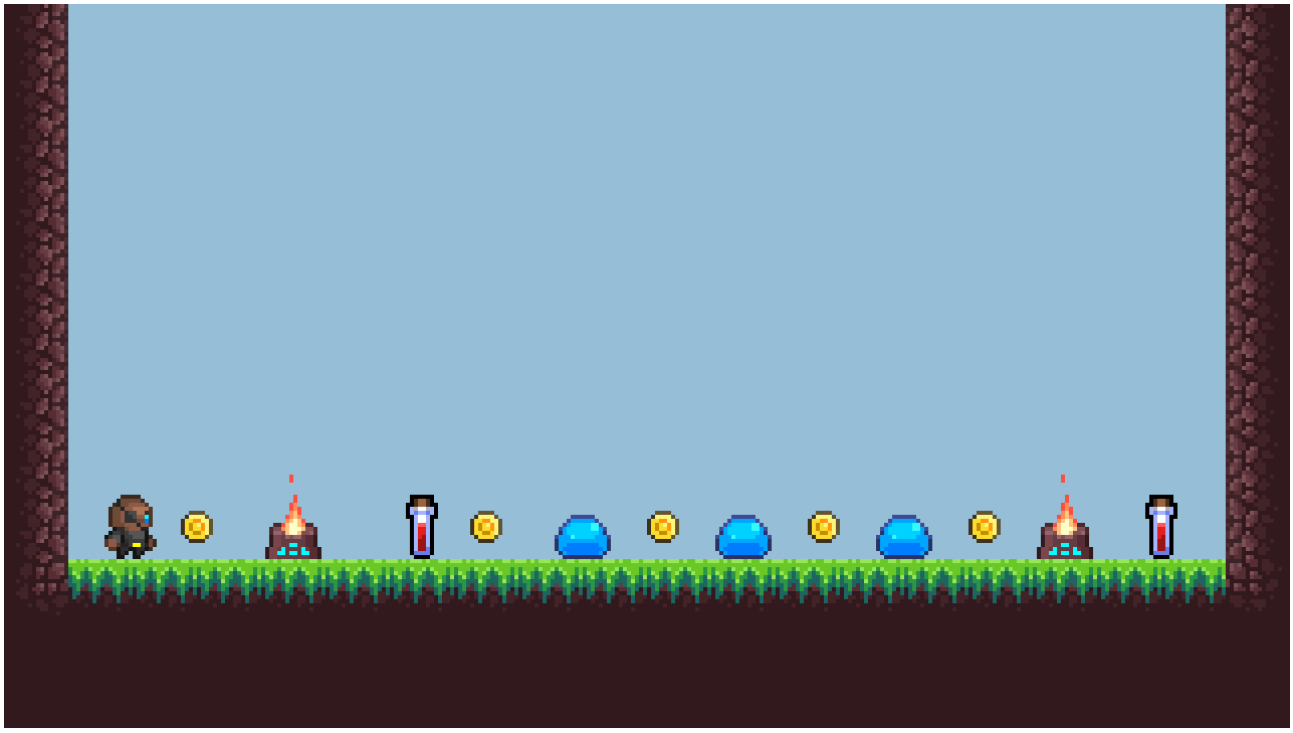


Figure 1: The finished scene: the player, coins, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S10_UIToolkit_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S10_UIToolkit_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S10_UIToolkit`.
3. Press **Play**. It's S03's room: fire, potions, J to attack and coins all work. The HUD is drawn, but it always says **5 / 5** and **Score: 0**, and **Restart** does nothing. Press Play again to stop.

The UI files are in `Assets/_game/UI/Toolkit`: `HUD.uxml`, `HUD.uss`, `PanelSettings` and `DefaultTheme`. The HUD's only GameObject is **HUD (UI Toolkit)**. You'll look at how it's built, then write the script that brings it to life.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Panel Settings

Select `PanelSettings` in `UI/Toolkit`. It does the Canvas Scaler's job from S03: **Scale Mode** is **Scale With Screen Size**, **Reference Resolution** 1920 x 1080, **Match** 0.5. Its **Theme Style Sheet** is `DefaultTheme`, the default look for buttons and labels. (To make your own: **Create > UI Toolkit > Panel Settings Asset**; Unity makes a theme with it.)

2. The layout: HUD.uxml

Double-click `HUD.uxml` to open it in the **UI Builder**. The **Hierarchy** on the left shows the elements; the **Viewport** shows what they look like. In a text editor, the file is:

```
<ui:UXML xmlns:ui="UnityEngine.UIElements">
    <Style src="HUD.uss" />
    <ui:VisualElement name="hud" class="hud" picking-mode="Ignore">
        <ui:VisualElement name="health-bar" class="health-bar">
            <ui:VisualElement name="health-fill" class="health-fill" />
            <ui:Label name="health-label" class="health-label" text="5 / 5" />
        </ui:VisualElement>
        <ui:Button name="restart-button" class="panel restart-button" text="Restart" />
        <ui:Label name="score-label" class="panel" text="Score: 0" />
    </ui:VisualElement>
</ui:UXML>
```

- **Elements are nested**, like GameObjects in the Hierarchy: `hud` holds `health-bar` (with `health-fill` and `health-label` inside), `restart-button` and `score-label`.
- **`name`** is how code finds an element. **`class`** is how styles find it. The button has two classes, `panel` and `restart-button`.
- **`<Style src="HUD.uss" />`** attaches the style sheet. **`picking-mode="Ignore"`** lets clicks pass through the empty parts of the HUD.

3. The looks: HUD.uss

USS is like CSS on the web. Open `HUD.uss` in your code editor and read through it; you don't need to change anything. (To make your own style sheet later: right-click in the Project window and choose **Create > UI Toolkit > Style Sheet**.)

```
/* Styles for HUD.uxml. USS is like CSS on the web: selectors pick elements,
   properties style them. */

/* The whole screen. Its children sit in a row: health bar left, button middle, score
   right. */
.hud {
    position: absolute;
    left: 0;
    top: 0;
    right: 0;
    bottom: 0;
    flex-direction: row;
    justify-content: space-between;
    align-items: flex-start;
    padding: 40px;
}

.health-bar {
    width: 420px;
    height: 56px;
    padding: 6px;
```

```
    background-color: #62181d;
    border-radius: 4px;
}

/* The script sets this element's width as a percentage of the bar. */
.health-fill {
    width: 100%;
    height: 100%;
    background-color: #c8323c;
}

/* Drawn on top of the bar, filling it, with the text centred. */
.health-label {
    position: absolute;
    left: 0;
    top: 0;
    right: 0;
    bottom: 0;
    color: white;
    font-size: 34px;
    -unity-text-align: middle-center;
}

/* Shared by the button and the score: a dark box with centred white text. */
.panel {
    width: 320px;
    height: 70px;
    margin: 0;
    background-color: #1e1820;
    color: white;
    font-size: 40px;
    -unity-text-align: middle-center;
    border-width: 0;
    border-radius: 4px;
}

.restart-button {
    width: 240px;
}

/* :hover and :active are pseudo-classes: the style while the mouse is over or
   pressing the button. */
.restart-button:hover {
    background-color: #3a3042;
}

.restart-button:active {
    background-color: #c8323c;
}
```

- **A selector** such as `.panel` picks every element with that class. The button and the score share `.panel`, so they look the same.
- **Flexbox, not anchors:** `.hud` fills the screen, lays its children in a `row`, and `justify-content: space-between` pushes them apart: health left, button centre, score right.
- **`:hover` and `:active`** style the button while the mouse is over it or pressing it.

Try it now: in the UI Builder, select `hud` and change **Flex > Direction** to **Column** in the Inspector. The HUD stacks vertically. Change it back (or undo with Ctrl+Z) without saving.

4. The UI Document

Select **HUD (UI Toolkit)** in the scene. Its **UI Document** component has **Panel Settings** set to `PanelSettings` and **Source Asset** set to `HUD`. This one component builds the whole HUD from the UXML: there are no GameObjects for the bar, the button or the score.

5. The code: ToolkitHud

Make a folder `Assets/_game/Scripts/UI` and create `ToolkitHud` in it:

```
// S03's HUD rebuilt with UI Toolkit. The layout is in HUD.uxml and the look in HUD.uss;
// this script finds the elements by name and fills them in.
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UIElements;

public class ToolkitHud : MonoBehaviour
{
    [SerializeField] private Health playerHealth;

    private VisualElement healthFill;
    private Label healthLabel;
    private Label scoreLabel;
    private Button restartButton;
    private int shownScore = -1;

    private void OnEnable()
    {
        // The UIDocument builds the elements from the UXML. Q ("query") finds one by
        // its name.
        VisualElement root = GetComponent<UIDocument>().rootVisualElement;
        healthFill = root.Q<VisualElement>("health-fill");
        healthLabel = root.Q<Label>("health-label");
        scoreLabel = root.Q<Label>("score-label");
        restartButton = root.Q<Button>("restart-button");

        // A UI Toolkit button has a C# event instead of an On Click list in the
        // Inspector.
        restartButton.clicked += RestartScene;
        playerHealth.HealthChanged += UpdateHealth;
    }

    private void OnDisable()
    {
        restartButton.clicked -= RestartScene;
        if (playerHealth != null)
        {
            playerHealth.HealthChanged -= UpdateHealth;
        }
    }

    private void Start()
    {
        UpdateHealth(playerHealth.CurrentHealth, playerHealth.MaxHealth);
    }

    private void Update()
    {
        if (GameManager.Instance != null && GameManager.Instance.Score != shownScore)
        {
```

```
            shownScore = GameManager.Instance.Score;
            scoreLabel.text = "Score: " + shownScore;
        }
    }

    private void UpdateHealth(int currentHealth, int maxHealth)
    {
        // Styles can be changed from code too. Length.Percent makes the width a
        // percentage of the bar.
        float percent = 100f * currentHealth / maxHealth;
        healthFill.style.width = Length.Percent(percent);
        healthLabel.text = currentHealth + " / " + maxHealth;
    }

    private void RestartScene()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }
}
```

What's going on:

- **`rootVisualElement`** is the top of the HUD built from the UXML. **`root.Q<Label>("score-label")`** ("query") finds the element with that name and type.
- **`restartButton.clicked += RestartScene;`** UI Toolkit buttons have no On Click list in the Inspector. You subscribe in code, like S09's C# event, and unsubscribe in `OnDisable`.
- **`HealthChanged`** is S09's event, so the bar only updates when the health changes. The score is checked every frame, like S03's ScoreDisplay.
- **`healthFill.style.width = Length.Percent(percent)`** changes a style from code: the fill's width becomes a percentage of the bar.

Add it to **HUD (UI Toolkit)** and drag the **Player** into **Player Health**.

Test it: press Play. Stand in the fire and the bar shrinks; collect coins and the score goes up. Hover over **Restart** (it gets lighter), press it (red while held), and the level restarts. Open S03 next to it to compare: same HUD, two different systems.

Check your work

- ☐ The bar and its text follow the player's health.
- ☐ The score goes up with each coin.
- ☐ Restart changes colour on hover and press, and reloads the level.
- ☐ Compare with `CRE132_S10_UIToolkit_Finished` if something behaves differently.

Key points

- **uGUI** (the Canvas, S03) is better documented, works for UI in the world, and is what most tutorials use.
- **UI Toolkit** separates layout (UXML), looks (USS) and logic (C#), and handles complex screens such as inventories and settings better.
- **Use one system per screen.** Mixing them in one project is fine; this course mostly uses uGUI.

If something goes wrong

- **NullReferenceException in ToolkitHud's OnEnable:** a name in `Q<>()` doesn't match the UXML (names are typed as strings, so a typo gives `null`), or the **Player Health** slot is empty.
- **"There is no 'UIDocument' attached":** ToolkitHud must go on the object with the **UI Document**.

- **The HUD has no colours and is stacked in the top left:** the `<Style src="HUD.uss" />` line in `HUD.uxml` has been changed or deleted, or `HUD.uss` has been renamed or moved out of `UI/Toolkit`.
- **Nothing shows at all:** the UI Document's **Panel Settings** or **Source Asset** is empty.

Try it

1. Change the colours in `HUD.uss` and watch both the button and the score change (they share the `.panel` class).
2. Add a `coins-label` to the UXML and fill it from code.
3. Add `transition-duration: 0.2s;` to `.health-fill` so the bar slides instead of jumping.